ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica - Scienza e Ingegneria - DISI

Corso di Laurea in Ingegneria e Scienze Informatiche

IMPLEMENTAZIONE CUDA DI UN ALGORITMO DI PROIEZIONE TOMOGRAFICA

Tesi di Laurea in Ingegneria e Scienze Informatiche

Relatore:
Prof. Moreno Marzolla

Correlatore:
Prof. Elena Loli Piccolomini

Presentata da:
Enrico Marchionni

Sessione marzo 2025

Anno Accademico 2024/2025

Sommario

La Tomografia Computerizzata (TC) è una tecnica diagnostica che sfrutta le radiazioni ionizzanti (o raggi X) per ottenere un'immagine tridimensionale di un'entità analizzata. Questa tecnica si basa sulla raccolta, da parte di un rilevatore, di una matrice di dati 2D. Tale matrice è costruita a partire da considerazioni fatte sull'attenuazione dei raggi X che, diretti da una sorgente verso il rilevatore, passano attraverso l'oggetto di studio. Facendo ruotare la sorgente e il rilevatore, lungo una traiettoria nota, molte matrici di dati vengono in genere raccolte. Queste matrici 2D vengono poi trasferite in un calcolatore che ha il compito di ricostruire l'entità scansionata in 3D. Questo consente di esaminare l'oggetto ricostruito, osservandone alcune caratteristiche della sua struttura interna non note a priori, senza doverlo a tal scopo necessariamente disassemblare.

Nonostante sfruttata principalmente in ambito medico, la teoria su cui si basa la TC, in realtà, non fu ideata per scopi medici. Infatti tale tecnica trova applicazione in numerosi altri ambiti oltre alla medicina, tra cui vari settori industriali, la paleontologia, l'archeologia e numerosi altri campi.

Il principio su cui si basa ha origine grazie al matematico austriaco Johann Radon, che nel 1917 dimostrò la possibilità di ricostruire un oggetto tridimensionale mediante un numero infinito di proiezioni bidimensionali dell'oggetto stesso. Dal punto di vista matematico, questo rappresenta un problema inverso, cioè si determinano le cause (immagine 3D) a partire dagli effetti osservati (proiezioni 2D). In questa tesi viene invece affrontato il problema diretto: partendo dal corpo studiato si devono perciò determinare le proiezioni che una TC potrebbe generare. Gli obiettivi di questo lavoro sono:

- Implementare una versione CUDA di un certo algoritmo di proiezione proposto da Siddon [Sid85] partendo da una sua implementazione parallela in C che sfrutta OpenMP [Col24].
- Analizzare le prestazioni della versione CUDA per GPU e confrontarle con la versione parallela per CPU già nota.

Indice

1 Introduzione **1**
1.1 Il modello matematico . . . 1
1.1.1 La legge di Lambert-Beer . . . 1
1.1.2 La trasformata di Radon . . . 2
1.1.3 Il caso discreto . . . 2
1.2 Algoritmo risolutivo . . . 3
1.2.1 La matrice M . . . 3
1.2.2 Il vettore f . . . 4
1.3 Considerazioni geometriche . . . 5
1.3.1 Posizionamento di sorgente, oggetto e rilevatore nello spazio . . . 5
1.3.2 Le sorgenti . . . 5
1.3.3 L'oggetto . . . 6
1.3.4 I rilevatori . . . 7
1.3.5 I raggi . . . 9

2 Descrizione del progetto **11**
2.1 Parallelizzazione . . . 11
2.1.1 CPU . . . 11
2.1.2 GPU . . . 12
2.2 Generazione input . . . 13
2.3 Calcolo proiezioni . . . 14
2.3.1 Implementazione generale . . . 16
2.3.2 Implementazione OpenMP . . . 17
2.3.3 Implementazione CUDA . . . 18
2.4 Generazione output . . . 21

3 Valutazione delle prestazioni **24**
3.1 Statistiche . . . 24
3.1.1 Tempo di esecuzione . . . 25
3.1.2 Throughput . . . 26
3.1.3 Speedup . . . 27

A Conclusioni **28**

Capitolo 1

Introduzione

Durante una scansione tomografica sono acquisite numerose proiezioni attorno all'oggetto di studio. Nel caso più semplice, che noi analizzeremo, la sorgente ed il rilevatore si muovono lungo una traiettoria circolare. Intuitivamente più sono le proiezioni acquisite, più la ricostruzione dell'oggetto sarà accurata. D'altra parte, specialmente in ambito medico, ridurre l'esposizione alle radiazioni è di beneficio per l'individuo scansionato. Questo implica che il numero di proiezioni generate è esso stesso molto variabile in base all'applicazione.

Dal punto di vista fisico, i dati di proiezione riflettono l'assorbimento dei fotoni che costituiscono i raggi X. Una proiezione può essere considerata come una matrice i cui valori rappresentano l'attenuazione dei raggi che attraversano il corpo oggetto di studio. Considerando la matrice proiezione, i valori rappresentano i vari livelli di attenuazione: un raggio che attraversa una regione più densa del corpo subisce una maggiore attenuazione rispetto ad un raggio che attraversa una regione meno densa.

Nelle sezioni successive verrà illustrato il modello matematico che costituisce la base della computazione, mettendolo in relazione diretta con le sue applicazioni nell'algoritmo risolutivo. Infine, saranno esaminate le considerazioni necessarie per risolvere il problema dal punto di vista geometrico.

1.1 Il modello matematico

Di seguito verranno discusse le nozioni matematiche fondamentali per comprendere, dal punto di vista algebrico, il calcolo alla base della creazione delle proiezioni, vedi [MP21].

1.1.1 La legge di Lambert-Beer

Il modello fisico che descrive l'assorbimento dei fotoni in termini di coefficienti di attenuazione è descritto dalla legge di Lambert-Beer.

Il meccanismo fisico che porta all'attenuazione dell'intensità di un raggio è solitamente descritto da un singolo coefficiente di attenuazione $\mu = \mu(w) \geq 0$ che dipende dalla posizione nel segmento che approssima il suo percorso, indicata da w. Questi coefficienti determinano i vari $m(w)$, cioè le intensità del raggio nella posizione indicata da w.

La **legge di Lambert-Beer** per il calcolo della proiezione del coefficiente di attenuazione lungo un segmento di lunghezza W è:

$$P_W \mu = -\ln\left(\frac{m}{m_0}\right) = \int_0^W \mu(w)\, dw \tag{1.1}$$

dove:

- $P_W \mu$ è un valore che indica l'attenuazione di un raggio; la notazione indica la *proiezione integrale* di μ lungo un segmento di lunghezza W.
- W è la lunghezza totale del percorso attraverso il materiale.
- $\mu(w)$ è una funzione continua che descrive il coefficiente di attenuazione alla posizione w lungo il raggio.
- m è l'intensità finale del raggio, dopo aver attraversato il materiale.
- m_0 è l'intensità iniziale del raggio.

1.1.2 La trasformata di Radon

La **trasformata di Radon**, in un modello continuo, è data dall'insieme delle proiezioni acquisite lungo l'intera traiettoria circolare. La rappresentazione grafica di tutti i dati misurati nel caso bidimensionale è chiamata *sinogramma*.

1.1.3 Il caso discreto

Nel caso discreto il corpo è diviso in volumi più piccoli, detti **voxel**, che sono elementi di dimensione molto piccola in cui l'oggetto è approssimato. Per ciascun voxel il coefficiente di attenuazione è un valore approssimato costante.

L'Equazione 1.1 nel caso discreto, per calcolare una singola proiezione, diventa:

$$g_i = \sum_{j=0}^{N} M_{i,j} f_j \quad \forall i \in 1, \ldots, N_p \tag{1.2}$$

dove:

- g_i è l'attenuazione subita dall'i-esimo raggio.
- i è l'i-esimo raggio, degli N_p considerati per quella proiezione.
- j è il j-esimo voxel, degli N totali che approssimano il corpo.
- $M_{i,j}$ è una matrice di $N_p \times N$ elementi che per ogni raggio i definisce la lunghezza della porzione sottesa al volume di ciascun voxel j.
- f_j è il valore del coefficiente di attenuazione assunto nel volume interno al j-esimo voxel.

L'indice j, considerando lo spazio cartesiano, è determinato dalle tre coordinate cartesiane, combinate tra di loro in modo da individuare un singolo indice.

Matematicamente quindi, per il calcolo delle proiezioni, viene considerata un'approssimazione dell'oggetto analizzato, vedi Figura 1.1.

Ritornando all'Equazione 1.2, si può notare come il numero totale di voxel N sia, considerando lo spazio cartesiano, determinata da $N_x \times N_y \times N_z$.

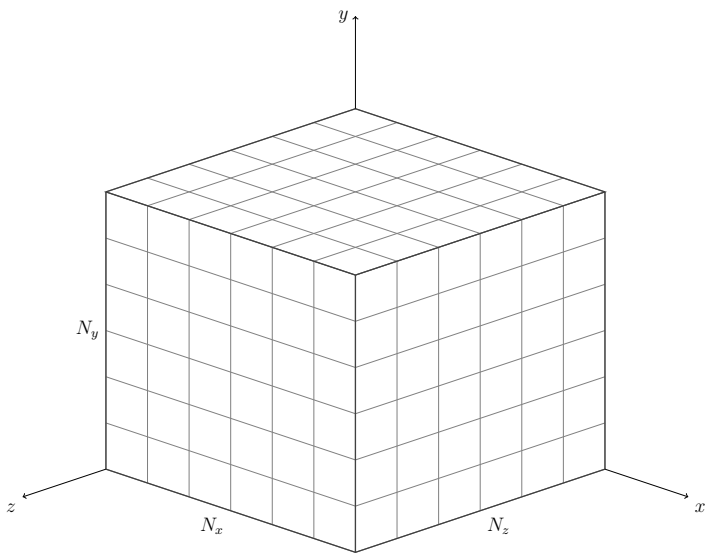


Figura 1.1: Esempio di oggetto cubico suddiviso in $N = N_x \times N_y \times N_z$ voxel cubici. Nel caso generale i voxel sono dei parallelepipedi retti, nel caso più semplice possono essere dei cubi con una dimensione $d = d_x = d_y = d_z$ fissata in base alle necessità.

Nel caso discreto della trasformata di Radon il numero di proiezioni dipende dal numero di posizioni considerate in cui si effettuano le scansioni, tale numero verrà di seguito indicato con N_θ. Ad ogni posizione la sorgente ed il rilevatore sono posti in modo da poter analizzare lo stesso oggetto da più angolazioni diverse.

1.2 Algoritmo risolutivo

Come già accennato in precedenza, alla base del progetto di tesi sviluppato in questa tesi c'è l'algoritmo di proiezione proposto da Siddon [Sid85].

Considerando quanto già visto nel modello matematico, nel nostro caso l'algoritmo di Siddon si occupa di calcolare la matrice M, mentre il vettore f deve essere già noto in partenza. Questi dati devono essere determinati per poter procedere con i passi successivi.

L'algoritmo svolge una computazione equivalente a quella descritta nell'Equazione 1.2:

$$g = \sum_i \sum_j \sum_k l_{i,j,k} \rho_{i,j,k} \tag{1.3}$$

dove gli indici i, j e k individuano un singolo voxel nello spazio tridimensionale. In particolare, per una specifica configurazione di (i, j, k), l rappresenta la lunghezza della porzione del segmento che approssima un raggio nello spazio ed è sottesa in quel voxel (equivalente di M) mentre ρ rappresenta la densità del voxel (equivalente di f).

1.2.1 La matrice M

La matrice M ci permette di identificare, per ogni raggio, la lunghezza del segmento di raggio che attraversa ciascun voxel.

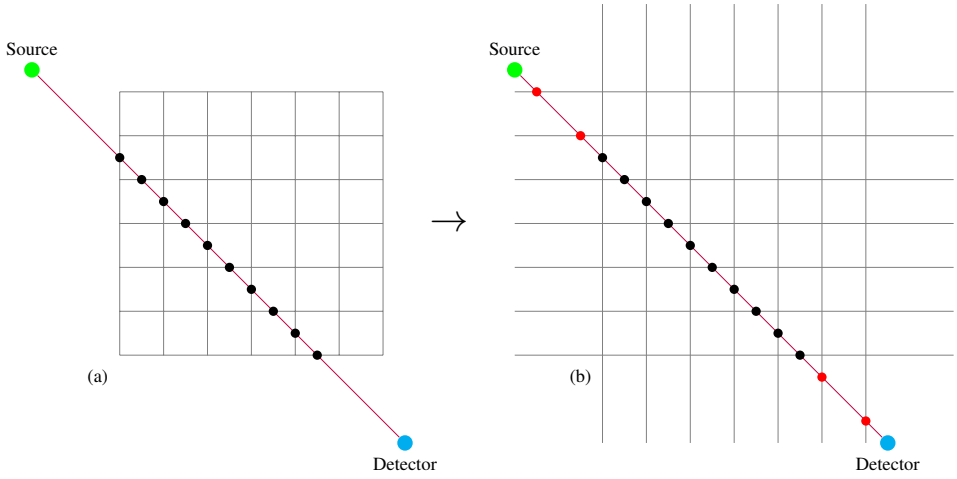


Figura 1.2: Pixel, caso 2D, considerati come rette parallele che rappresentano la struttura dell'oggetto considerato. Nell'algoritmo si parlerà invece di voxel e piani paralleli, caso 3D. Come si può notare alcune intersezioni tra il raggio e i piani non sono valide perché fuori dalla griglia, vedi i punti rossi della figura *b*.

I punti chiave dell'algoritmo, nella determinazione della matrice M, possono essere così riassunti:

- L'algoritmo considera, invece dei singoli voxel indipendenti, tre insiemi di piani ortogonali ed equidistanti tra loro che vanno a costruire la griglia di discretizzazione del corpo nello spazio, vedi Figura 1.2.
- Si calcolano i punti di intersezione tra le rette, cioè i raggi, e i piani che definiscono i voxel.
- Si calcola la lunghezza dei segmenti sottesi ai voxel, come la differenza tra punti di intersezione consecutivi, determinando così la matrice M.

1.2.2 Il vettore f

Il vettore f riassume ciò che sappiamo dell'oggetto da scansionare. In particolare esso rappresenta il coefficiente di attenuazione assunto nella regione interna al volume occupato da ciascun voxel.

I coefficienti di attenuazione nel progetto di tesi sviluppato sono: 1 per una regione "molto densa", approssimando sempre uno stesso tipo di materiale, 0 per una "poco densa", approssimando l'aria.

I tre oggetti considerati sono:

- **Cubo**: stabilita la lunghezza del lato del cubo che si intende rappresentare, gli elementi di f all'interno della regione cubica sono posti ad 1, altrimenti a 0.
- **Cubo con cavità sferica**: stabilita la lunghezza del lato del cubo che si intende rappresentare, gli elementi di f all'interno della regione cubica, ad eccezione di quelli la cui distanza da un punto stabilito all'interno dell'oggetto sia inferiore ad un raggio stabilito, sono posti ad 1, altrimenti a 0.
- **Semisfera**: stabilita la lunghezza del raggio della semisfera che si intende rappresentare, gli elementi di f all'interno della regione semisferica sono posti ad 1, altrimenti a 0.

1.3 Considerazioni geometriche

Di seguito si approfondiscono i riferimenti geometrici necessari per poter procedere con l'implementazione dell'algoritmo di Siddon.

1.3.1 Posizionamento di sorgente, oggetto e rilevatore nello spazio

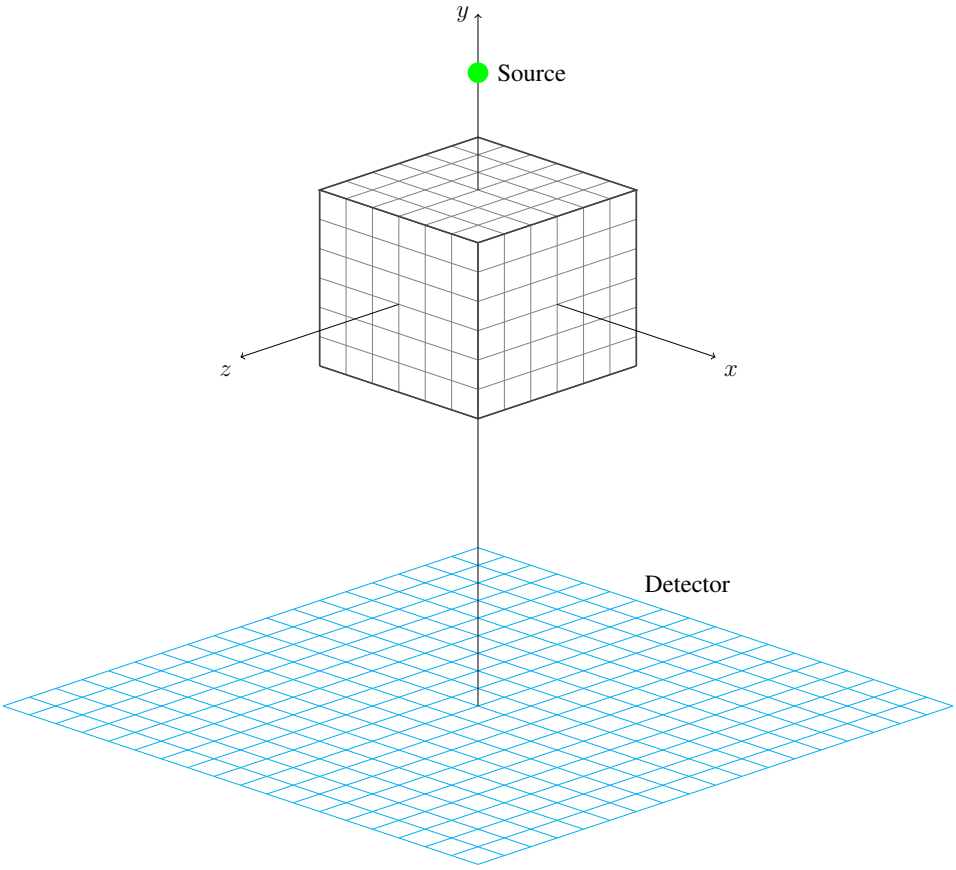


Figura 1.3: Esempio che rappresenta l'oggetto cubico già visto in Figura 1.1 nel sistema di riferimento cartesiano considerato nelle discussioni seguenti.

In Figura 1.3 sono rappresentati un oggetto di forma cubica, la sorgente ed il rilevatore, tutti posti in uno specifico sistema cartesiano in cui il centro dell'oggetto corrisponde con l'origine degli assi.

La sorgente dei raggi e i pixel del rilevatore, verso cui sono diretti, ruotano lungo un'orbita circolare per scansionare il corpo da diverse angolazioni.

Quando la sorgente ruota in un senso, orario o antiorario, fino a specifiche posizioni nell'orbita, il rilevatore ruota coerentemente, quindi dello stesso angolo, nel senso opposto.

1.3.2 Le sorgenti

La sorgente è approssimata ad un punto, che indica la posizione a partire dalla quale sono diretti i raggi. Le posizioni considerate sono più di una e, nell'implementazione trattata, sono

distribuite lungo una traiettoria circolare (o una sezione di essa) intorno all'oggetto. Occorre stabilire le coordinate cartesiane dei punti che rappresentano le sorgenti nello spazio.

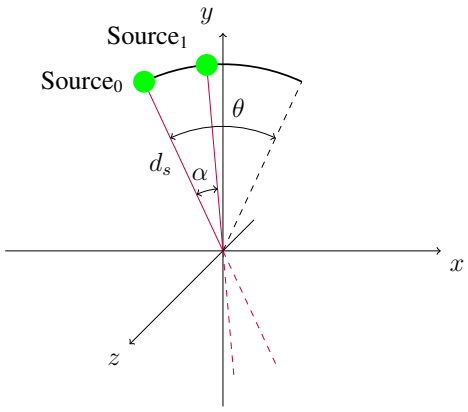


Figura 1.4: Posizionamento angolare delle prime due sorgenti.

Conoscendo l'estensione angolare θ della sezione della traiettoria considerata e la distanza angolare α tra ciascuna posizione, si è scelto l'uso delle coordinate polari, dove una posizione è individuata dalla distanza dall'origine e dalla distanza angolare dall'asse polare. In accordo con la Figura 1.4, viene considerato un sistema di coordinate polari il cui polo coincide con il centro dell'oggetto di studio, mentre si adotta l'asse y come asse polare. La distanza angolare è positiva in senso orario nel piano xy.

La posizione della sorgente può essere individuata nel seguente modo:

$$(d_s, -\theta/2 + \alpha i) \quad \text{per } i = 0, \ldots, N_\theta - 1 \tag{1.4}$$

dove:

- i è un indice che identifica una posizione della sorgente.
- d_s è la distanza sorgente-origine.
- N_θ è il numero di posizioni dato dal rapporto $N_\theta = \lfloor \theta/\alpha \rfloor + 1$.

Le coordinate cartesiane si ottengono a partire dalle coordinate polari nel seguente modo:

$$\begin{cases} X_i = \cos(\frac{\pi}{2} - (-\frac{\theta}{2} + \alpha i))d_s = \sin(-\frac{\theta}{2} + \alpha i)d_s \\ Y_i = \sin(\frac{\pi}{2} - (-\frac{\theta}{2} + \alpha i))d_s = \cos(-\frac{\theta}{2} + \alpha i)d_s \\ Z_i = 0 \end{cases} \qquad \text{per } i = 0, \ldots, N_\theta - 1 \tag{1.5}$$

dove sono state applicate le formule degli archi associati per seno e coseno per passare dal sistema con misurazione in senso orario a partire dal semiasse positivo delle y al sistema tradizionale con misurazione in senso antiorario a partire dal semiasse positivo delle x. Inoltre, Z_i è posta a 0 in quanto, per costruzione, la traiettoria delle sorgenti giace sul piano xy.

1.3.3 L'oggetto

Un oggetto è approssimato da un certo numero di voxel, cioè parallelepipedi retti con dimensioni uniformi l'uno con l'altro. Le griglie che questi creano, dall'algoritmo di Siddon, sono rappresentate da un certo numero di piani paralleli tra loro.

L'equazione cartesiana di un piano nello spazio è un'equazione di primo grado in tre incognite:

$$ax + by + cz + d = 0 \quad \text{con } a, b, c, d \in \Re$$

Avendo in mente la Figura 1.3, i piani coinvolti, oltre che ad essere paralleli tra di loro, sono paralleli anche agli assi cartesiani. Dunque, le loro equazioni sono nella forma:

$$\begin{aligned} ax + d = 0 &\quad \text{se parallelo al piano } yz \\ by + d = 0 &\quad \text{se parallelo al piano } xz \\ cz + d = 0 &\quad \text{se parallelo al piano } xy \end{aligned}$$

Dato che le distanze tra i piani paralleli sono uniformi, avendo l'equazione di un piano è possibile determinare l'equazione degli altri traslando più volte. L'equazione degli altri piani a partire dal primo si determina nel seguente modo:

$$\begin{aligned} X_{plane}(i) &= X_{plane}(1) + (i-1)d_x \quad \forall i \in 1, \ldots, N_x \\ Y_{plane}(j) &= Y_{plane}(1) + (j-1)d_y \quad \forall j \in 1, \ldots, N_y \\ Z_{plane}(k) &= Z_{plane}(1) + (k-1)d_z \quad \forall k \in 1, \ldots, N_z \end{aligned}$$

dove N_x, N_y e N_z sono il numero di voxel lungo gli assi x, y e z, mentre d_x, d_y e d_z sono a loro volta le dimensioni di un singolo voxel lungo i tre assi cartesiani.

1.3.4 I rilevatori

Nel caso tridimensionale, il rilevatore può essere visto come una matrice di unità di misura, ossia i pixel, posizionata a una distanza d_r dal centro del corpo in esame.

L'approccio usato per relazionare i raggi con il rilevatore è detto *ray-driven*. Per ciascun unità del rilevatore si considera l'esistenza di un unico raggio passante, diretto dalla sorgente al centro del pixel stesso. Secondo questo approccio, il numero N_p di raggi usati per il calcolo della proiezione coincide con il numero di pixel del rilevatore.

Il rilevatore è sempre ortogonale all'asse che congiunge la sorgente con il centro del corpo.

Le coordinate cartesiane del centro di un'unità del rilevatore, al variare dell'angolo considerato, sono (X', Y', Z'), derivate dalla posizione (X, Y, Z), ricoperta nel caso in cui il rilevatore sia ortogonale all'asse y.

Considerando il semiasse negativo di y come asse polare, anche questa volta con angolo positivo in senso orario, le coordinate polari del punto sono:

$$(d_r, \delta)$$

dove δ è l'angolo che identifica l'unità del rilevatore.

Le coordinate a seguito della traslazione saranno:

$$(d_r, \delta - \lambda)$$

dove $-\lambda$ è l'angolo di traslazione.

Le coordinate di tale posizione traslata sono date da:

$$\begin{cases} X' = \cos(\frac{3\pi}{2} - (\delta - \lambda))d_r = -\sin(\delta - \lambda)d_r \\ Y' = \sin(\frac{3\pi}{2} - (\delta - \lambda))d_r = -\cos(\delta - \lambda)d_r \end{cases} \tag{1.6}$$

dove, in modo analogo all'Equazione 1.5, sono state applicate le formule degli archi associati per seno e coseno per passare dal sistema con misurazione in senso orario a partire dal semi asse negativo delle y al sistema tradizionale con misurazione in senso antiorario a partire dal semi asse positivo delle x. In questo caso Z è costante, $Z' = Z$, in quanto, per costruzione, la traiettoria trasla sul piano xy, senza influire sulla coordinata nell'asse z, quindi verrà omessa nella discussione seguente.

A partire dall'Equazione 1.6, applicando le formule di addizione degli archi per seno e coseno si ottiene:

$$\begin{cases} X' = -(\sin(\delta)\cos(-\lambda)d_r + \cos(\delta)\sin(-\lambda)d_r) \\ Y' = -(\cos(\delta)\cos(-\lambda)d_r - \sin(\delta)\sin(-\lambda)d_r) \end{cases}$$

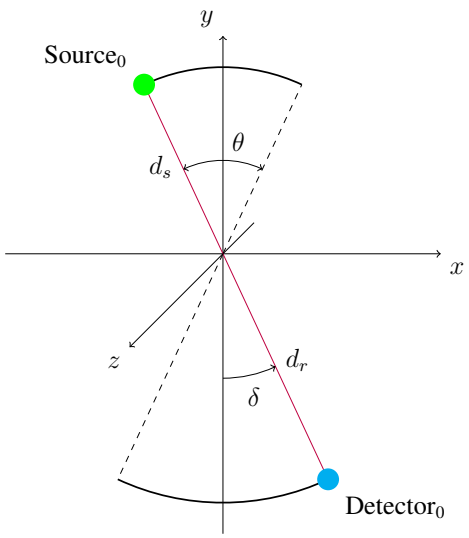


Figura 1.5: Vista angolare di un raggio diretto dalla prima sorgente verso il centro del pixel centrale nel rilevatore. Gli angoli corrispondono esattamente a quelli nell'esempio quando la matrice che costituisce il rilevatore ha un numero di pixel per lato dispari.

Osservando la Figura 1.5, dalle formule trigonometriche per i triangoli rettangoli:

$$d_r = -\frac{X}{\sin(\delta)} = -\frac{Y}{\cos(\delta)}$$

sostituendo d_r e sfruttando le formule degli archi associati per seno e coseno si ottiene:

$$\begin{cases} X' = \cos(-\lambda)X + \sin(-\lambda)Y \\ Y' = \cos(-\lambda)Y - \sin(-\lambda)X \end{cases}$$

Analogamente a quanto avviene con la sorgente, la posizione di una unità del rilevatore varia, ad intervalli regolari, di un angolo α su di una sezione di ampiezza regolare pari a θ. Quindi, ricordando l'Equazione 1.4, si considera $-\lambda = -\theta/2 + \alpha i$. Le posizioni di un'unità del

rilevatore possono dunque essere ottenute, a partire dalla posizione precedente, con:

$$\begin{cases} X'_i = \cos(-\theta/2 + \alpha i)X + \sin(-\theta/2 + \alpha i)Y \\ Y'_i = \cos(-\theta/2 + \alpha i)Y - \sin(-\theta/2 + \alpha i)X \end{cases} \qquad \text{per } i \in 0, \ldots, N_\theta - 1$$

1.3.5 I raggi

Un raggio, approssimazione di radiazioni elettromagnetiche, è considerato come un segmento che, nell'implementazione usata, collega la sorgente con il centro di un pixel del rilevatore (approccio ray-driven). Può essere rappresentato mediante l'equazione parametrica di una retta passante per due punti $Source = (X_1, Y_1, Z_1)$ e $Detector = (X_2, Y_2, Z_2)$:

$$\begin{aligned} X(a) &= X_1 + a(X_2 - X_1) \\ Y(a) &= Y_1 + a(Y_2 - Y_1) \\ Z(a) &= Z_1 + a(Z_2 - Z_1) \end{aligned}$$

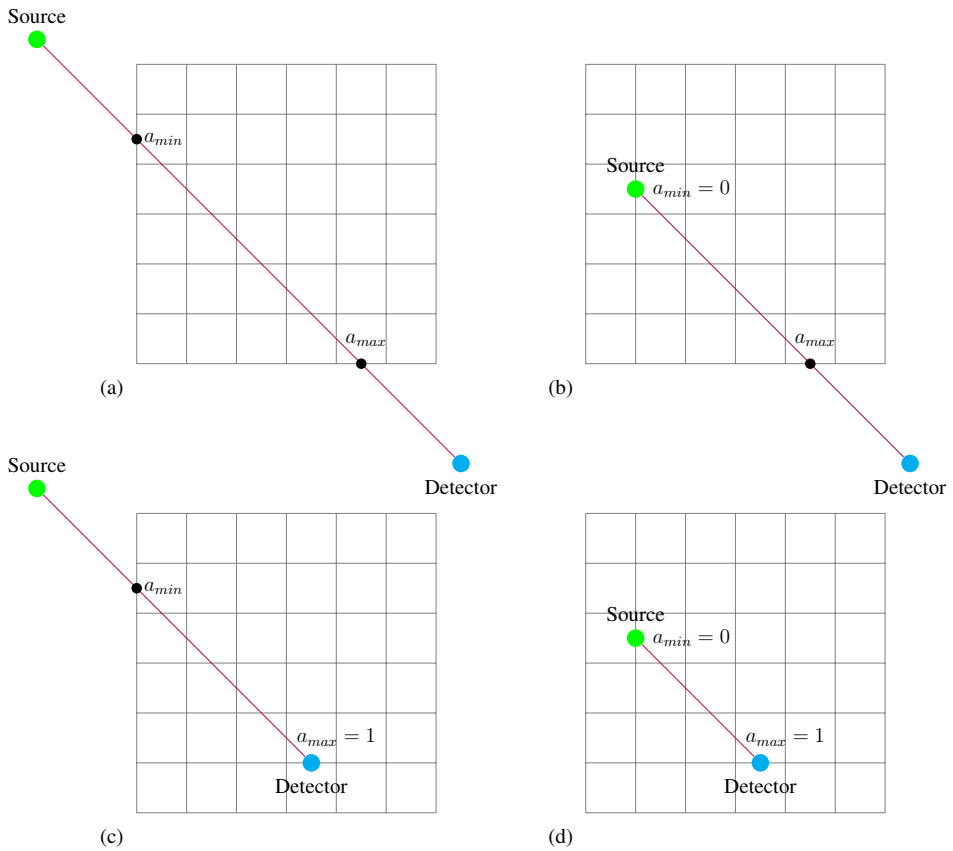


Figura 1.6: Le quantità a_{min} e a_{max} sono quelle che definiscono gli intervalli da considerare per ottenere solo le intersezioni con piani effettivamente all'interno della griglia di pixel (voxel nel caso 3D). Si può notare che a_{min} nelle figure *a* e *c* sarà maggiore di 0, così come a_{max} nelle figure *a* e *b* sarà minore di 1. Questo permette di non considerare le intersezioni in eccesso, esterne alla griglia di partenza, risolvendo il problema esposto nella Figura 1.2.

In questo modo:

- Per definire un raggio basta conoscere il punto "sorgente" ed il punto sul "rilevatore" dove avviene la misurazione.
- È possibile rappresentare un punto appartenente alla retta del raggio attraverso un unico parametro a, in particolare $a = 0$ coincide con il punto $Source$, $a = 1$ coincide con il punto $Detector$, per i valori di a compresi tra $[0, 1]$ si hanno punti interni al segmento, vedi Figura 1.6.

Il punto di intersezione di un raggio ed un piano si ottiene risolvendo in a il sistema lineare dato dall'equazione del piano e una componente della retta come di seguito:

$$\begin{cases} X = X_1 + a(X_2 - X_1) \\ X = X_{planes}(1) + (i-1)d_x \end{cases} \qquad \text{se } i \in 1, \ldots, N_x$$

$$\begin{cases} Y = Y_1 + a(Y_2 - Y_1) \\ Y = Y_{planes}(1) + (i-1)d_y \end{cases} \qquad \text{se } i \in 1, \ldots, N_y$$

$$\begin{cases} Z = Z_1 + a(Z_2 - Z_1) \\ Z = Z_{planes}(1) + (i-1)d_z \end{cases} \qquad \text{se } i \in 1, \ldots, N_z$$

dove sono stati indicati i sistemi utilizzati nel caso in cui il piano sia parallelo al piano yz (primo sistema), al piano xz (secondo sistema) o al piano xy (terzo sistema).

Capitolo 2

Descrizione del progetto

Il progetto di tesi include diverse implementazioni scritte in C, tra cui un programma per la generazione dei file di input, uno per il calcolo della proiezione basato sull'algoritmo di Siddon con OpenMP ed una versione sviluppata con CUDA.

Le prime due soluzioni sono state ottenute rielaborando il lavoro di [Col24], mentre l'implementazione CUDA costituisce il fulcro di questo progetto.

Di seguito verrà riportata una breve introduzione alle tecniche di parallelizzazione utilizzate, a cui seguirà la descrizione delle implementazioni sviluppate.

2.1 Parallelizzazione

L'ottimizzazione di un algoritmo seriale è un processo complesso, che coinvolge molteplici strategie a seconda del contesto applicativo, delle caratteristiche hardware e degli obiettivi di prestazione. In generale, l'ottimizzazione può riguardare la riduzione della complessità computazionale, una migliore gestione della memoria, l'uso di strutture dati efficienti o l'impiego di tecniche avanzate per minimizzare i tempi di esecuzione.

Tuttavia, quando si passa ad implementazioni ottimizzate per architetture specifiche come CPU e GPU, emergono strategie di ottimizzazione mirate, che sfruttano le peculiarità di ciascun tipo di hardware.

Nel seguito, ci concentreremo sulle tecniche di parallelizzazione adottate nelle implementazioni su CPU e GPU selezionate, analizzando le metodologie impiegate per massimizzare le prestazioni in ciascun caso.

2.1.1 CPU

La CPU (Central Processing Unit) è il componente principale di un computer responsabile dell'esecuzione delle istruzioni di un programma. Essa rappresenta l'unità di controllo ed elaborazione principale, in quanto gestisce i calcoli, l'elaborazione dei dati ed il controllo delle operazioni.

Le CPU moderne, per motivi principalmente legati al risparmio energetico, dispongono di più core e supportano il multi-threading, consentendo l'esecuzione parallela di più attività.

OpenMP

OpenMP[1] (Open Multi-Processing) è un'API (Application Programming Interface) che supporta la programmazione parallela su architetture multi core con memoria condivisa. Fornisce un insieme di direttive per il compilatore, librerie runtime e variabili d'ambiente che permettono di scrivere programmi paralleli. In questo progetto è stato utilizzato il linguaggio C.

OpenMP rientra nella categoria **MIMD** (Multiple Instruction, Multiple Data) secondo la tassonomia di Flynn, vedi [Fly66].

Caratteristiche principali di OpenMP:

- Parallelismo a memoria condivisa: I processi condividono lo stesso spazio di indirizzamento.
- Programmazione basata su direttive: Le sezioni di codice parallelo sono specificate tramite pragma.
- Scalabilità: Adatta il numero di processi in base al numero di core disponibili.
- Facilità di utilizzo: Non richiede riscrittura completa del codice, ma solo l'aggiunta di annotazioni.
- Sincronizzazione e gestione del carico: Include meccanismi per bilanciare il carico tra i processi ed evitare race conditions.

2.1.2 GPU

La GPU (Graphics Processing Unit) è un processore specializzato progettato per eseguire calcoli in parallelo su grandi quantità di dati. Come si può intuire dal nome è stata originariamente sviluppata per il rendering grafico. Conseguentemente alla domanda del mercato, la GPU si è oggi ampiamente evoluta in un processore generale, che quindi può svolgere molti diversi tipi di carichi di lavoro al di là del solo rendering grafico. Utilizzata in settori come intelligenza artificiale, elaborazione di immagini, deep learning e simulazioni scientifiche, grazie alla sua capacità di eseguire migliaia di operazioni simultaneamente. Le GPU moderne, sebbene operino in generale a frequenze più basse delle CPU, sono molto più veloci di esse nell'eseguire i compiti in cui sono specializzate.

CUDA

CUDA[2] (Compute Unified Device Architecture) è una piattaforma di calcolo parallelo e un modello di programmazione sviluppato da NVIDIA che consente di sfruttare le GPU NVIDIA per l'elaborazione generale, GPGPU (General-Purpose computing on Graphics Processing Units). In questo progetto è stato utilizzato il linguaggio 'CUDA-C' (C con estensioni NVIDIA).

A livello CUDA, per lavorare in modo efficiente, si astrae dal funzionamento hardware, considerando un parallelismo di tipo **SIMT** (Single Instruction, Multiple Thread), vedi [Lin+08], che è un'estensione del concetto SIMD (Single Instruction, Multiple Data) della tassonomia di Flynn, vedi [Fly66].

In realtà l'hardware delle GPU prevede esecuzione:

- SIMT, a livello di Warp (unità di schedulazione):

[1] Per maggiori informazioni vedi `https://www.openmp.org/`.
[2] Per maggiori informazioni vedi `https://docs.nvidia.com/cuda/`.

  - Le GPU eseguono gruppi di Thread chiamati Warp (tipicamente 32 Thread) che eseguono la stessa istruzione simultaneamente, ma su dati diversi. Questo è il comportamento tipico del paradigma SIMD.
  - Tuttavia, a differenza di un vero SIMD (dove tutti i dati sono rigidamente legati alla stessa istruzione), i Thread in un Warp possono, anche se inefficiente, divergere (ad esempio, con istruzioni condizionali), motivo per cui il modello è chiamato SIMT.
- MIMD, a livello di multiprocessore:
  - Ogni Streaming Multiprocessor (SM) della GPU può eseguire più Warp indipendenti, ognuno con istruzioni diverse.
  - Questo significa che, considerando l'intera GPU, essa opera in modo MIMD, poiché più SM possono eseguire istruzioni diverse su set di dati diversi.

Il meccanismo di computazione basato sulle unità Warp implica che le computazioni SIMT offrano le migliori prestazioni. Questo perché le divergenze, ovvero le istruzioni condizionali nel codice, causano la serializzazione dell'esecuzione dei Thread all'interno di un singolo Warp.

Caratteristiche principali di CUDA:

- Parallelismo massivo: Permette di eseguire migliaia di Thread in parallelo sulla GPU. Adatto a risolvere problemi del tipo 'Embarrassingly Parallel', quindi in cui la computazione può essere decomposta in task indipendenti, con poche divergenze e che richiedono poca comunicazione l'uno con l'altro.
- Memoria gerarchica: Struttura di memoria ottimizzata per alte prestazioni (memoria globale, condivisa, registri).
- Elevata efficienza: Permette di accelerare calcoli scientifici, AI, machine learning, rendering, simulazioni fisiche, ecc.

2.2 Generazione input

L'implementazione proposta genera le configurazioni del vettore f già discusse nella sottosezione 1.2.2. Queste configurazioni vengono scritte su file. Per generare input di dimensioni opportune alle necessità, quando si esegue il programma è possibile specificare il numero di pixel per lato della matrice rilevatore. La strategia adottata per la scrittura è descritta di seguito.

```
unsigned Nx = N_VOXEL_X, Nz = N_VOXEL_Z, pNy = k;
// Dove k <= N_VOXEL_Y
for (unsigned y = 0; y < pNy; y++) {
  for (unsigned z = 0; z < Nz; z++) {
    for (unsigned x = 0; x < Nx; x++) {
      f[x + z*Nz + y*Nx*Nz] = 1.0;
    }
  }
}
```

Listato 2.1: Codice C per inizializzazione di f per un cubo.

Il Listato 2.1 mostra l'inizializzazione semplificata di f per un oggetto cubico denso.

Per comprendere l'indicizzazione è utile considerare che l'elemento nell'array 1D $f[x + zN_z + yN_xN_z]$, è equivalente a quello nella matrice 3D $\rho[x][z][y]$, in cui x, z ed y sono fissati, rivedi l'Equazione 1.3.

La caratteristica fondamentale di questa soluzione è che l'inizializzazione mostrata, di fatto, "srotola" la matrice ρ in un array f, in cui la coordinata y è indipendente rispetto alle altre. In questo senso le x e poi le z sono scritte più velocemente rispetto alla y, quindi dipendono da essa. In altre parole, la y, nel codice al Listato 2.1, è l'unico contatore non annidato, di conseguenza quello che non dipende dagli altri.

Si osserva però che la specifica mappatura degli elementi scelta non è l'unica possibile, si sarebbero per esempio potute scambiare le varie coordinate tra di loro, invertendone i ruoli, e perciò le relative dipendenze.

Il Listato 2.1 rappresenta in realtà un'iterazione dell'inizializzazione di f. Infatti `pNy` non è assegnato a `N_VOXELS_Y`, cioè numero totale di voxel nell'asse y, ma a `k`. Quindi l'array f, limitato nelle dimensioni da `pNy`, rappresenta una partizione verticale dell'oggetto cubico di interesse e non l'oggetto completo così come considerato nel modello matematico, rivedi la Figura 1.3. Tale partizione avrà numero di voxel sull'asse y pari a `k`, con `k <= N_VOXELS_Y`. Nella soluzione proposta oltretutto la singola iterazione, simile a quella presentata, viene parallelizzata sfruttando OpenMP.

In questo modo, anche la lettura dell'input può essere fatta per partizioni sul numero totale di voxel nell'asse y. È essenziale riconoscere che tali partizioni possono assumere dimensioni arbitrarie, senza vincoli reciproci e, in particolare, senza necessità di conoscere quella scelta per la scrittura.

Questa strategia di scrittura è essenziale per gestire input di grandi dimensioni. In particolare, nella computazione su CPU, è fondamentale considerare la quantità di memoria RAM disponibile sulla scheda madre, un fattore rilevante nella parallelizzazione con OpenMP. Nella computazione su GPU, invece, bisogna tenere conto sia della memoria RAM della scheda madre sia di quella disponibile sulla GPU, aspetto cruciale quando si parallelizza con CUDA. Ad esempio, supponiamo di dover analizzare una partizione di lunghezza `yVoxels`, con la condizione `yVoxels <= N_VOXELS_Y`. In questo caso, per quella partizione verranno letti solo `N_VOXEL_X * yVoxels * N_VOXEL_Z` valori, il che, per input di grandi dimensioni, può comportare un notevole risparmio di RAM.

2.3 Calcolo proiezioni

L'algoritmo per il calcolo delle proiezioni implementato si basa su quanto discusso nel Capitolo 1, in particolare tieni a mente l'Equazione 1.2 da cui siamo partiti. I passi dell'algoritmo sono i seguenti:

1. il vettore f dei coefficienti di attenuazione viene letto da file come input del programma;
2. si calcola il prossimo punto sorgente da analizzare, nel caso in cui siano finiti l'algoritmo termina;
3. si calcola il prossimo pixel del rilevatore (in realtà il punto centrale), nel caso in cui siano finiti si ritorna al punto 2;
4. per procedere si calcolano i valori parametrici di intersezione del raggio individuato dalla sorgente e dal pixel del rilevatore con il primo e l'ultimo piano di ogni dimensione, per verificare che il raggio attraversi l'oggetto, in caso contrario si ritorna al punto 3;

5. a questo punto si calcolano i valori parametrici dei piani estremi (il primo e l'ultimo) che devono essere considerati per calcolare il sottoinsieme delle intersezioni con il raggio che risultano dentro la griglia dei voxel, rivedi la Figura 1.2;
6. poi si calcolano i valori parametrici delle intersezioni tra il raggio ed i piani considerati;
7. si calcola la lunghezza del prossimo segmento sotteso ai voxel (valore di M), nel caso in cui siano finiti si ritorna al punto 3;
8. ora si calcola l'indice del voxel che sottende il segmento;
9. al pixel del rilevatore (pixel corrente di g) viene infine sommata l'attenuazione del voxel corrispondente, moltiplicata per la lunghezza del segmento considerato, si ritorna quindi al punto 7.

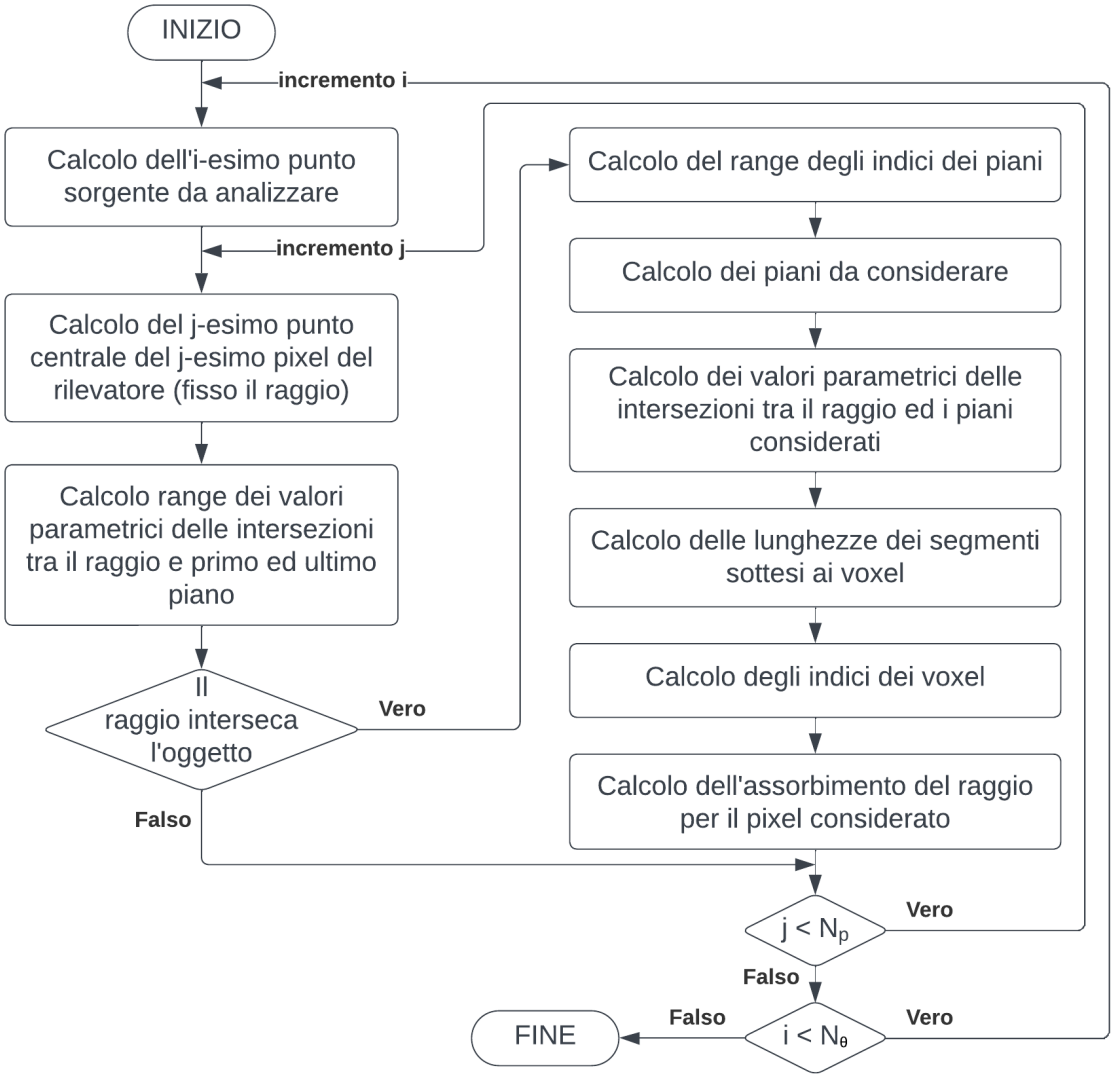


Figura 2.1: Diagramma di flusso del calcolo delle proiezioni.

In Figura 2.1 è presentato il diagramma di flusso relativo ai punti 1-9 dell'algoritmo di calcolo delle proiezioni.

2.3.1 Implementazione generale

1. Lettura input

```
double *const f = (double *) malloc(sizeof(double) * gl_nVoxel[X] *
    yVoxels * gl_nVoxel[Z]);
...
for (unsigned short slice = 0; slice < gl_nVoxel[Y]; slice +=
    yVoxels) {
  unsigned short nOfSlices;
  if (gl_nVoxel[Y] - slice < yVoxels) {
    nOfSlices = gl_nVoxel[Y] - slice;
  } else {
    nOfSlices = yVoxels;
  }
  fread(f, sizeof(double), (size_t) gl_nVoxel[X] * nOfSlices *
      gl_nVoxel[Z], inputFilePointer);
  ...
  computeProjections(...);
}
```

Listato 2.2: Codice C per la lettura di f.

Il Listato 2.2 mostra la gestione dell'array f, che può costituire l'intero insieme di voxel o un suo sottoinsieme. La lettura dei dati viene quindi fatta considerando `nOfSlices` voxel alla volta sull'asse y, dove `yVoxels` è il numero di voxel considerati in una singola iterazione sull'asse y. Nella funzione `computeProjections` sono sviluppati tutti i calcoli necessari per ottenere le proiezione. Tutti i passi descritti successivamente sono svolti all'interno di tale funzione.

2. Individuazione della prossima sorgente

```
for (unsigned short positionIndex = 0; positionIndex < nTheta;
    positionIndex++) {
  const Point source = getSource(gl_sinTable, gl_cosTable,
      positionIndex);
  ... // Calcolo di ogni pixel del rilevatore
}
```

Listato 2.3: Codice C per l'individuazione di un punto sorgente.

Il Listato 2.3 mostra come avviene il calcolo di ogni punto sorgente in base all'angolo di rotazione. Il calcolo di ogni punto sorgente è ripetuto per ogni partizione considerata. I dettagli della funzione `getSource` non sono mostrati, il calcolo è lo stesso già presentato nella sottosezione 1.3.2.

3. Individuazione del prossimo pixel del rilevatore

```
... // Per ogni sorgente
for (unsigned r = 0; r < nSidePixels; r++) {
  for (unsigned c = 0; c < nSidePixels; c++) {
    ...
```

```
    const Point pixel = getPixel(gl_sinTable, gl_cosTable, r, c,
        positionIndex);
    ... // Calcolo dell'assorbimento per il pixel considerato
  }
}
```

Listato 2.4: Codice C per l'individuazione di un pixel del rilevatore.

Il Listato 2.4 mostra come avviene il calcolo di ogni punto centrale di un pixel del rilevatore in base all'angolo di rotazione. I dettagli della funzione `getPixel` non sono mostrati, il calcolo è lo stesso già presentato nella sottosezione 1.3.4.

4-9. Computazioni successive

Le computazioni seguenti sono eseguite facendo riferimento all'algoritmo di Siddon [Sid85].

2.3.2 Implementazione OpenMP

L'implementazione per CPU proposta sfrutta il parallelismo OpenMP.

Il calcolo della soluzione, per motivi legati all'utilizzo di memoria RAM, viene fatto in partizioni, considerando di default un numero massimo di `100` voxel sull'asse y.

Nella rielaborazione dell'implementazione di partenza è stata introdotta la possibilità di specificare il numero massimo di voxel quando si lancia il programma da linea di comando, ignorando quindi il valore di default.

Parallelizzazione

In questo contesto, nonostante alcune modifiche, il modello di parallelizzazione è stato mantenuto invariato rispetto a quello del codice originale.

```
... // Per ogni sorgente
#pragma omp parallel for collapse(2) schedule(dynamic) ...
for (unsigned r = 0; r < nSidePixels; r++) {
  for (unsigned c = 0; c < nSidePixels; c++) {
    ... // Calcolo dell'assorbimento per il pixel considerato
  }
}
```

Listato 2.5: Codice C per il calcolo dell'assorbimento di un determinato pixel del rilevatore.

Il Listato 2.5 mostra le direttive OpenMP usate per parallelizzare il codice presentato nel Listato 2.4. Ogni processo OpenMP determina la posizione di un pixel del rilevatore e procede al calcolo dell'attenuazione che si verifica quando il raggio attraversa la partizione dell'oggetto in esame. La parallelizzazione viene eseguita considerando un carico di lavoro molto sbilanciato tra i diversi processi. Infatti il numero di computazioni necessarie, per ogni raggio relativo ad una sorgente, potrebbe portare ad un carico di lavoro molto disomogeneo.

È stata mantenuta la stessa versione di partenza che adottava un partizionamento a grana fine seguendo il paradigma master-worker, in cui:

- Partizionamento a grana fine: si riferisce a una suddivisione del lavoro in compiti molto piccoli, assegnati ai vari processi per massimizzare il parallelismo e bilanciare il carico. Nel nostro caso l'unità di parallelizzazione è il singolo valore fissato di `r` e `c`.
- Paradigma master-worker: è un modello di programmazione parallela in cui un processo master si occupa di assegnare i compiti ai processi worker. I worker eseguono i compiti assegnati e, una volta completato un task, passano immediatamente al successivo disponibile, seguendo l'ordine di completamento. Questo approccio consente una distribuzione dinamica ed efficiente del carico di lavoro, ottimizzando l'utilizzo delle risorse computazionali.

2.3.3 Implementazione CUDA

L'implementazione per GPU NVIDIA proposta sfrutta il parallelismo CUDA.

Analogamente alla versione OpenMP, la soluzione viene calcolata per partizioni, al fine di ottimizzare l'uso della memoria RAM. Tuttavia, in questo caso, il numero massimo di voxel lungo l'asse y viene determinato dinamicamente mediante un algoritmo euristico. Questo algoritmo stima rapidamente la memoria occupata sulla GPU, fornendo un valore approssimativo che potrebbe non essere necessariamente ottimale.

```
size_t freeMem, totalMem;
cudaMemGetInfo(&freeMem, &totalMem);
freeMem -= (totalMem * 2 / 8);
unsigned tmp = gl_nVoxel[Y];
size_t size = sizeof(double) * gl_nVoxel[X] * tmp * gl_nVoxel[Z];
while (size > freeMem) {
  tmp = tmp * 5 / 8;
  if (tmp == 0) {
    fprintf(stderr,
      "The voxels Y size is too small respect to the other sizes:\n"
      "- N voxels X: %u.\n"
      "- N voxels Y: %u.\n"
      "- N voxels Z: %u.\n"
      "Total size reduced to the minimum possible is %lu Bytes!\n"
      "This is too much for this GPU with %lu Bytes of usable "
      "global memory (of %lu Bytes total)!\n",
      gl_nVoxel[X], gl_nVoxel[Y], gl_nVoxel[Z],
      size, freeMem, totalMem);
    termEnvironment();
    exit(EXIT_FAILURE);
  }
  size = sizeof(double) * gl_nVoxel[X] * tmp * gl_nVoxel[Z];
}
yVoxels = tmp;
```

Listato 2.6: Codice CUDA-C per il calcolo del numero massimo di voxel sull'asse y.

Il Listato 2.6 mostra il funzionamento dell'algoritmo per il calcolo del numero massimo di voxel sull'asse y per ogni partizione dell'input da considerare. Dopo la sua individuazione il valore calcolato è assegnato a `yVoxels`.

Anche in questo caso è stata introdotta la possibilità di specificare il numero massimo di voxel quando si lancia il programma da linea di comando, ignorando quindi il valore generato dall'algoritmo euristico che agisce normalmente.

Parallelizzazione

Nelle discussioni seguenti, il termine host si riferirà all'esecuzione di codice seriale sulla CPU, mentre device indicherà l'esecuzione di codice parallelo sulla GPU.

```
double *const f = (double *) malloc(sizeof(double) * gl_nVoxel[X] *
    yVoxels * gl_nVoxel[Z]);
...
for (unsigned short slice = 0; slice < gl_nVoxel[Y]; slice +=
    yVoxels) {
  unsigned short nOfSlices;
  if (gl_nVoxel[Y] - slice < yVoxels) {
    nOfSlices = gl_nVoxel[Y] - slice;
  } else {
    nOfSlices = yVoxels;
  }
  fread(f, sizeof(double), (size_t) gl_nVoxel[X] * nOfSlices *
      gl_nVoxel[Z], inputFilePointer);
  ...
  asyncComputeProjections(...);
}
```

Listato 2.7: Codice CUDA-C per la lettura di f.

Il Listato 2.7, equivalente del Listato 2.2, mostra come viene svolta la computazione considerando partizioni dell'input.

In questo caso, a differenza di OpenMP, la funzione `computeProjections` non viene invocata direttamente, bensì viene chiamata un'altra funzione, `asyncComputeProjections`. Questa scelta è motivata dalla necessità di garantire coerenza con il tipo di computazione eseguita al suo interno.

Si noti che il prefisso `async` nella funzione `asyncComputeProjections`, chiamata dall'host, indica che l'esecuzione avviene in modo asincrono. Così si permette alla CPU di leggere simultaneamente la prossima partizione dell'input, introducendo un ulteriore livello di parallelismo tra le operazioni.

Tale meccanismo è reso possibile dalla separazione tra la memoria RAM della CPU e quella della GPU. Di conseguenza, una volta copiati i dati di input nella RAM della GPU, la CPU può immediatamente sovrascrivere il buffer contenente la partizione appena trasferita, subito dopo aver avviato il kernel in modalità asincrona. Inoltre, nel caso in cui non ci siano più dati da essere letti allora il buffer di lettura può essere deallocato, in tal caso l'host, inevitabilmente, attenderà il completamento del kernel per poter leggere i risultati ottenuti.

Questo approccio ha un impatto significativo sulla misurazione dei tempi di esecuzione: se il numero di iterazioni è elevato e la durata dell'esecuzione del kernel non è inferiore a quella della lettura dell'input da file, è come se dal tempo totale di esecuzione dei vari kernel (escluso l'ultimo) venisse sottratto il tempo di lettura da file. In pratica, ciò si traduce in una riduzione complessiva dei tempi di esecuzione.

```
void asyncComputeProjections(...)
{
  // Si copia f dalla CPU nell'array corrispondente d_f sulla GPU
  cudaMemcpy(d_f, f, sizeF, cudaMemcpyHostToDevice);
  // Le dimensioni di griglia e blocco sono sempre le stesse
  static dim3 block(BLKDIM_STEP, BLKDIM_STEP);
  static dim3 grid((nSidePixels + BLKDIM_STEP - 1) / BLKDIM_STEP, (
     nSidePixels + BLKDIM_STEP - 1) / BLKDIM_STEP);
  // Il kernel viene lanciato in modo asincrono
  computeProjections<<<grid, block>>>(slice, nTheta, nSidePixels,
     d_f, d_g, isFirst);
}
```

Listato 2.8: Codice CUDA-C per il lancio del kernel.

Il Listato 2.8 mostra l'inizializzazione del kernel CUDA. Il kernel `computeProjections`, che si occupa del calcolo delle proiezioni, non è più eseguito nell'host, ma nel device. Quindi l'esecuzione dei passi 2-9 dell'algoritmo considerato, cioè del kernel, avviene in parallelo sul device.

```
__global__ void computeProjections(...)
{
  ...
  // r e c mappano ogni Thread su un singolo pixel del rilevatore
  const unsigned r = threadIdx.y + blockIdx.y * blockDim.y;
  const unsigned c = threadIdx.x + blockIdx.x * blockDim.x;

  // Gli eventuali Thread in eccesso non eseguono calcoli
  if (r < nSidePixels && c < nSidePixels) {
    // Calcolo dell'attenuazione di una coppia sorgente-pixel
    for (unsigned short positionIndex = 0; positionIndex < nTheta;
       positionIndex++) {
      const Point source = getSource(d_gl_sinTable, d_gl_cosTable,
         positionIndex);
      const Point pixel = getPixel(d_gl_sinTable, d_gl_cosTable, r,
         c, positionIndex);
      ... // Calcolo dell'assorbimento per il pixel considerato
    }
  }
}
```

Listato 2.9: Codice CUDA-C del kernel.

Nel Listato 2.9 viene mostrato come ogni CUDA Thread determini la posizione di un pixel del rilevatore per procedere al calcolo dell'attenuazione che si verifica quando il raggio attraversa la partizione dell'oggetto in esame.

Nel ciclo illustrato, ogni CUDA Thread analizza ciascuna sorgente in riferimento ad uno specifico pixel, che è lo stesso per ogni iterazione sulle posizioni sorgente-rilevatore considerate. Tale pixel è però ricalcolato di volta in volta perché traslato di un certo angolo.

Il vantaggio di questa soluzione è che viene creato un solo kernel per ogni partizione dell'input, garantendo così il numero minimo possibile di esecuzioni. Questo riduce al minimo l'overhead legato alla creazione dell'ambiente di esecuzione del kernel. Inoltre, i dati necessari per il calcolo ad eccezione dell'input, che deve essere aggiornato per ogni partizione, vengono trasferiti tra CPU e GPU una sola volta.

2.4 Generazione output

Il risultato dell'algoritmo di proiezione è l'array g di valori in virgola mobile, si faccia riferimento all'Equazione 1.2. Tali valori vengono mappati su scala di grigi, facilitando la conversione in formato `.pgm`, per poter poi essere visualizzati come immagini, convertendoli in formati visualizzabili graficamente come `.png`.

```
double *const g = (double *) calloc(nSidePixels * nSidePixels *
    nTheta, sizeof(double));
... // Calcolo di g con OpenMP o CUDA
FILE *const outputFilePointer = fopen(outputFileName, "w");
if (!outputFilePointer) {
  fprintf(stderr, "Unable to open file '%s'!\n", outputFileName);
  free(g);
  return EXIT_FAILURE;
}
// Stampa un valore nell'intervallo [0-255]
fprintf(outputFilePointer, "P2\n%d %d\n255", nSidePixels,
    nSidePixels * nTheta);
for (unsigned short positionIndex = 0; positionIndex < nTheta;
    positionIndex++) {
  double angle = -(double) gl_angularTrajectory / 2 + (double)
      positionIndex * gl_positionsAngularDistance;
  fprintf(outputFilePointer, "\n#%lf", angle);
  for (unsigned i = 0; i < nSidePixels; i++) {
    fprintf(outputFilePointer, "\n");
    for (unsigned j = 0; j < nSidePixels; j++) {
      const unsigned pixelIndex = positionIndex * nSidePixels *
          nSidePixels + i * nSidePixels + j;
      int color = (g[pixelIndex] - gMinValue) * 255 / (gMaxValue -
          gMinValue);
      fprintf(outputFilePointer, "%d ", color);
    }
  }
}
fclose(outputFilePointer);
```

Listato 2.10: Codice C per la mappatura dell'output in immagini.

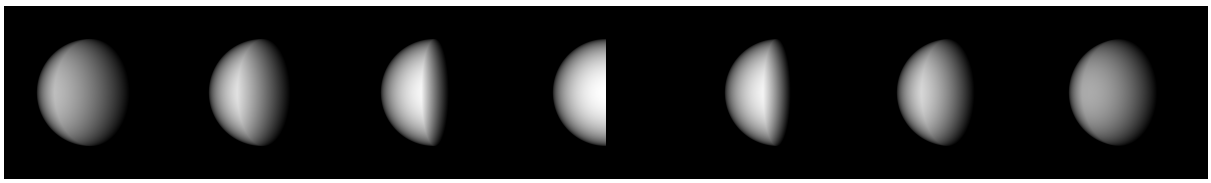
Il Listato 2.10 mostra come avviene la mappatura dei valori di g in un intervallo di valori 0-255 che individuano i colori nero-bianco nella scala di grigi.

In un'immagine generata a partire dalla matrice di proiezione, i colori rappresentano visivamente i diversi livelli di attenuazione. Un raggio che attraversa una regione più densa del corpo subirà una maggiore attenuazione, risultando in una tonalità più chiara (verso il bianco), mentre

un raggio che attraversa una regione meno densa subirà una minore attenuazione, apparendo in una tonalità più scura (verso il nero).

(a) Cubo.

(b) Cubo con cavità sferica.



(c) Semisfera.

Figura 2.2: Immagini delle proiezioni.

La Figura 2.2 mostra le immagini in formato `.png` delle proiezioni ottenute eseguendo la versione CUDA del programma. Si ricorda che le immagini ottenute non sono fotografie tradizionali dell'oggetto scansionato, piuttosto sono una rappresentazione della distribuzione di densità dell'oggetto scansionato. In questo caso l'input processato è stato generato specificando `2000` pixel per lato del detector, questo comporta che ciascuna proiezione qui mostrata è costituita da `2000x2000` pixel in scala di grigi. In particolare le immagini riportate, lette da sinistra a destra, corrispondono alle proiezioni del rispettivo oggetto ottenute considerando angoli di -45°, -30°, -15°, 0°, 15°, 30° e 45° in cui come asse polare si intende l'asse y e il senso orario di rotazione delle sorgenti. Le proiezioni sono state calcolate considerando come angoli di posizionamento della sorgente rispetto all traiettoria circolare $\theta = 90°$ e $\alpha = 15°$, rivedi Figura 1.4.

È importante inoltre considerare che, a causa delle differenze nelle architetture hardware di CPU e GPU, unitamente al fatto che l'ordine dei l'ordine dei calcoli in virgola mobile eseguiti in OpenMP e CUDA può variare, i risultati ottenuti non saranno esattamente identici. Per confrontare i risultati delle due versioni del programma e verificarne la correttezza, si possono adottare due approcci:

- Comparare direttamente le versioni binarie dei risultati, scrivendo la matrice g su file. Questo è il metodo più accurato.
- Comparare le versioni `.pgm` dei risultati, dove gg viene mappata da una matrice di valori in virgola mobile a un'altra di valori nell'intervallo 0-255. Questo metodo è meno accurato.
- Comparare le versioni `.png` dei risultati ad occhio. Questo metodo è il meno accurato.

Per i risultati generati, oltre alla verifica visiva, cioè il terzo metodo, come conferma è stato utilizzato il secondo metodo. Su Linux, questo confronto può essere effettuato tramite il comando:

```
compare -metric RMSE PGM1 PGM2 OUTPUT
```

dove:

- `PGM1` va rimpiazzato con il percorso del primo file in formato `.pgm` da considerare.
- `PGM2` va rimpiazzato con il percorso del secondo file in formato `.pgm` da considerare.
- `OUTPUT` va rimpiazzato con il percorso del file in cui verranno scritte le differenze tra i due file considerati.

Si deve inoltre considerare che se la metrica RMSE (Root Mean Square Error) ottenuta è un valore sotto a `1000` allora i due risultati possono essere considerati molto simili.

Capitolo 3

Valutazione delle prestazioni

Nella valutazione delle prestazioni per l'implementazioni CUDA trattata in questo progetto di tesi, le metriche di analisi considerate sono il tempo di esecuzione, il throughput e lo speedup rispetto alla versione OpenMP.

Si inizierà analizzando la complessità computazionale dell'algoritmo implementato, per poi presentare le statistiche ottenute dai test pratici eseguiti su un server.

Sia per OpenMP che per CUDA è stata utilizzata la versione di default dei programmi. Per OpenMP si considera un numero massimo di `100` voxel sull'asse y, mentre per CUDA questo valore è determinato dall'algoritmo euristico descritto in precedenza.

3.1 Statistiche

Prima di analizzare i risultati ottenuti è utile individuare la complessità computazionale dell'algoritmo considerato. Il limite superiore del tempo di esecuzione dell'algoritmo viene già individuato nella tesi di Colletta [Col24]. Tale complessità computazionale è qui riproposta:

$$O(N_\theta \cdot N_{pixel}^2 \cdot (N_x + N_y + N_z)) \tag{3.1}$$

dove:

- O detta o-grande, indica che si tratta di un limite superiore del tempo di esecuzione, relativo al caso in cui un raggio interseca tutti i piani.
- N_θ è il numero di proiezioni calcolate, cioè il numero di posizioni della sorgente considerate.
- N_{pixel} è il numero di unità di misurazione per lato del rilevatore.
- N_x è il numero di voxel lungo l'asse x.
- N_y è il numero di voxel lungo l'asse y.
- N_z è il numero di voxel lungo l'asse z.

Il server considerato nei test riportati successivamente è l'`isi-raptor03.csr.unibo.it`. La CPU del server è un `Intel(R)Xeon(R)CPU E5-2603 v4 @ 1.70GHz` con 12 core fisici e 12 core logici (quindi senza Hyper-Threading). La scheda madre è dotata di 64 GB di RAM. La GPU usata nel server è una `NVIDIA GeForce GTX 1070`. La scheda grafica è dotata di 8 GB di RAM.

Gli input considerati sono i file `CubeWithSphericalHole{400-1500}.dat`, cioè del tipo: cubo con cavità sferica. Il numero di pixel per lato della matrice rilevatore varia da `400` a `1500` con intervalli di `100` pixel tra un input ed il successivo, per un totale di 12 input diversi.

Iterazione	N_θ	N_{pixel}	N_x	N_y	N_z
1	7	400	168		
2	7	500	210		
3	7	600	252		
4	7	700	294		
5	7	800	336		
6	7	900	378		
7	7	1000	420		
8	7	1100	462		
9	7	1200	504		
10	7	1300	546		
11	7	1400	588		
12	7	1500	630		

Tabella 3.1: Dimensioni del problema per ogni iterazione considerata.

La Tabella 3.1 mostra quali sono le dimensioni del problema. N_θ è costante perché identificato dagli angoli fissati $\theta = 90°$ e $\alpha = 15°$. Il numero di voxel sui vari assi N_x, N_y e N_z sono uguali perché viene considerato un oggetto cubico. Tale valore del numero di voxel per asse viene calcolato dal generatore di input in base al numero di pixel per lato del rilevatore scelti per ogni iterazione in modo empirico in base all'hardware a disposizione, N_{pixel}.

3.1.1 Tempo di esecuzione

Il tempo di esecuzione potrebbe essere semplicemente considerato come il tempo totale impiegato dal programma per completare l'elaborazione. Tuttavia, questo includerebbe anche i tempi di lettura dell'input e di scrittura dell'output, influenzando significativamente la misurazione. Per questo motivo, nel seguito verrà considerato il tempo di esecuzione dato da:

- Allocazione dati: per la CPU semplicemente l'allocazione delle strutture dati e la loro opportuna inizializzazione, per la GPU anche il trasferimento dalla RAM dell'host a quella del device.
- Esecuzione dei calcoli dei valori di attenuazione per ogni chiamata alla funzione OpenMP `computeProjections` o a quella CUDA `asyncComputeProjections`.
- Deallocazione dati: per la CPU semplicemente la deallocazione delle strutture dati, per la GPU anche il trasferimento dalla RAM del device a quella dell'host.

Non verranno considerati:

- Tempo di esecuzione della lettura da file dell'input, perché le operazioni sono le stesse in entrambe le versioni.

- Tempo di esecuzione della scrittura su file dell'output, perché anche se il risultato delle due versioni è leggermente diverso a causa dei calcoli in virgola mobile, le operazioni eseguite sono le stesse.

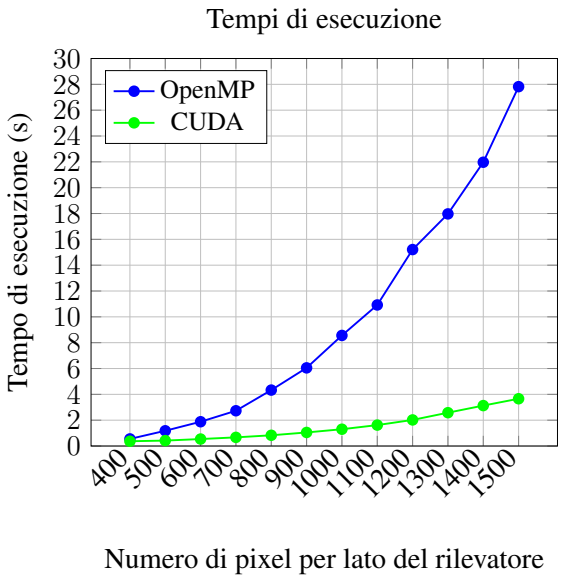


Figura 3.1: Tempi di esecuzione di CUDA ed OpenMP a confronto.

La Figura 3.1 mostra i tempi di esecuzione a confronto per OpenMP e CUDA. Come si può notare i tempi di esecuzione della versione CUDA sono fino alle 7 volte inferiori rispetto a quelli della versione OpenMP.

3.1.2 Throughput

Il throughput, è il numero di operazioni al secondo in funzione della dimensione dell'input. La dimensione dell'input è approssimata con il lavoro svolto dall'algoritmo, vedi l'Equazione 3.1.

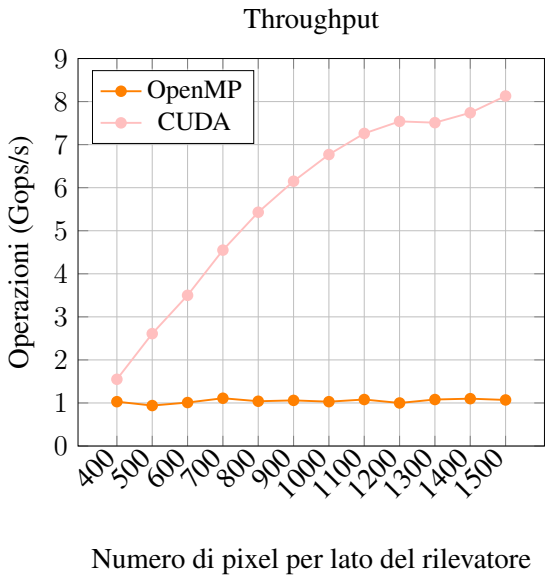


Figura 3.2: Throughput di CUDA ed OpenMP a confronto.

Il throughput è calcolato come $\frac{Numero\ di\ operazioni}{Tempo\ di\ esecuzione\ (s)}$. Per stimare il limite superiore del numero di operazioni al secondo svolte dai programmi sono stati utilizzati i dati riportati nella Tabella 3.1, i risultati sono osservabili nella Figura 3.2. Tale numero di istruzioni è approssimato a miliardi di operazioni per secondo. In questo caso, il numero di operazioni al secondo per CUDA arriva fino a 8x rispetto a quello della versione OpenMP.

3.1.3 Speedup

Lo speedup di CUDA è inteso come il confronto dei tempi di esecuzione della versione CUDA rispetto a quella OpenMP. Indica quanto l'implementazione per GPU è mediamente più veloce rispetto a quella per la CPU, al variare della dimensione dell'input.

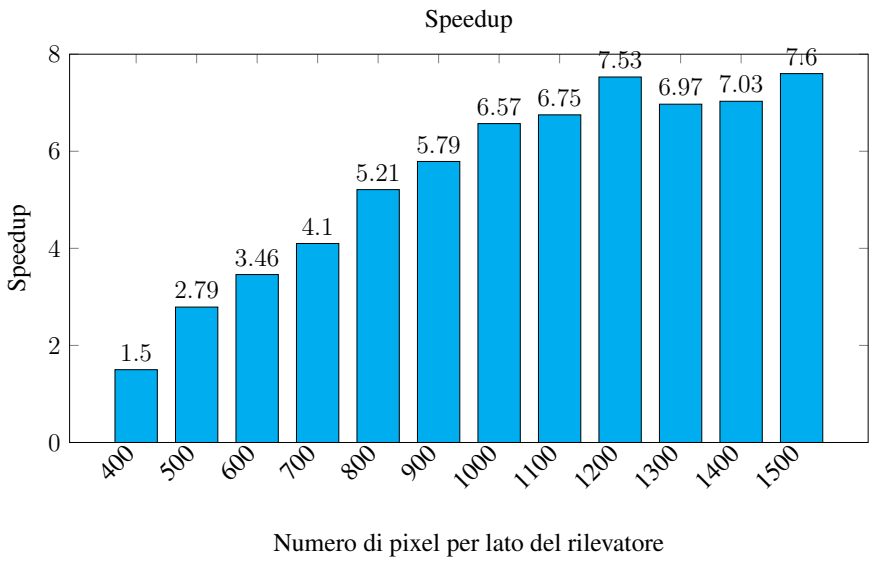


Figura 3.3: Accelerazione di CUDA rispetto ad OpenMP.

La Figura 3.3 mostra le performance della versione CUDA rispetto a quella OpenMP. Per input piccoli, l'accelerazione della versione CUDA è limitata, mentre cresce con la dimensione, stabilizzandosi intorno a 6x, con un picco di 7.6x in un caso favorevole.

Appendice A

Conclusioni

...

Bibliografia

[Col24] Lorenzo Colletta. «Implementazione parallela di un algoritmo di proiezione tomografica». Tesi di laurea triennale. Ingegneria e Scienze Informatiche, Università di Bologna, Cesena, 2024.

[MP21] Elena Morotti e Elena Loli Piccolomini. «Sparse Regularized CT Reconstruction: An Optimization Perspective». In: *Handbook of Mathematical Models and Algorithms in Computer Vision and Imaging: Mathematical Imaging and Vision*. A cura di Ke Chen et al. Cham: Springer International Publishing, 2021, pp. 1–34. ISBN: 978-3-030-03009-4. DOI: 10.1007/978-3-030-03009-4_123-1. URL: https://doi.org/10.1007/978-3-030-03009-4_123-1.

[Lin+08] Erik Lindholm et al. «NVIDIA Tesla: A Unified Graphics and Computing Architecture». In: *IEEE Micro* 28.2 (2008), pp. 39–55. DOI: 10.1109/MM.2008.31.

[Sid85] Robert L. Siddon. «Fast calculation of the exact radiological path for a three-dimensional CT array». In: *Medical Physics* 12.2 (1985), pp. 252–255. DOI: https://doi.org/10.1118/1.595715. eprint: https://aapm.onlinelibrary.wiley.com/doi/pdf/10.1118/1.595715. URL: https://aapm.onlinelibrary.wiley.com/doi/abs/10.1118/1.595715.

[Fly66] M.J. Flynn. «Very high-speed computing systems». In: *Proceedings of the IEEE* 54.12 (1966), pp. 1901–1909. DOI: 10.1109/PROC.1966.5273.